

JAVA LAB RECORD

Experiment No: 05

Date: 15/09/2026

Title: Demonstration of OOP Concepts in Java

■ Objective / Aim

To write and execute a Java program demonstrating major Object-Oriented Programming concepts including constructors, encapsulation, inheritance, polymorphism, abstraction, and interfaces.

■ System Requirements

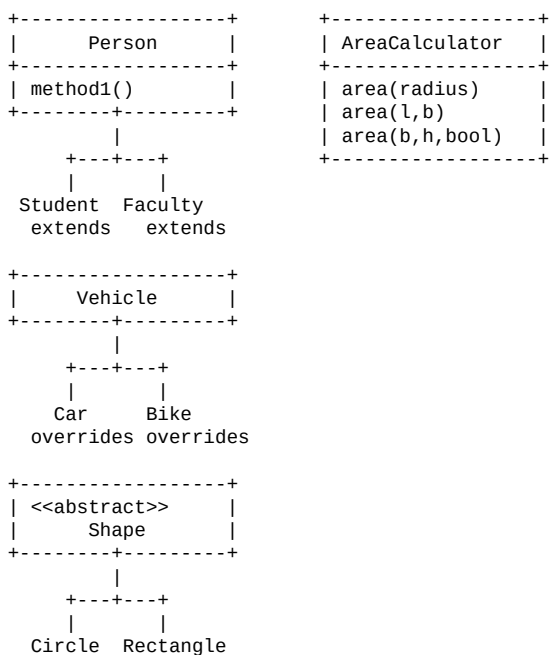
Hardware: Standard PC/Laptop, Core i3/i5 Processor or equivalent, 4 GB/8 GB RAM.

Software: Windows/Linux/macOS, JDK 17/JDK 21, Eclipse, IntelliJ IDEA, NetBeans, VS Code, or Command Prompt/Terminal.

■ Algorithm / Logic Description

1. Create a Book class with book ID, book name, author, and price.
2. Use a parameterized constructor in Book to initialize the values and method2() to display them.
3. Create Book1 with private variables, default and parameterized constructors, getters, setters, and details() to demonstrate encapsulation.
4. Create Person as a superclass and Student and Faculty as subclasses to demonstrate inheritance.
5. Create AreaCalculator with multiple area() methods having different parameter lists to demonstrate method overloading.
6. Create Vehicle and override display() in Car and Bike to demonstrate runtime polymorphism.
7. Create abstract Shape with draw(), then implement draw() in Circle and Rectangle.
8. Create Printable interface with print(), and implement it in Report.
9. In main(), create objects and invoke their methods to display the results.

■ UML Class Diagram



draw() draw()

```
+-----+
| <<interface>> |
| Printable      |
+-----+
```

|
Report
implements

```
+-----+
|      Book      |
+-----+
| bookId         |
| bookName       |
| author         |
| price          |
+-----+
| Book(...)      |
| method2()      |
+-----+
```

```
+-----+
|      Book1     |
+-----+
| - bookId       |
| - bookName     |
| - author       |
| - price        |
+-----+
| Book1()        |
| Book1(...)     |
| getters/setters|
| details()      |
+-----+
```

■ Source Code — Your Exact Code

```
class Book {
    int bookId;
    String bookName;
    String author;
    double price;

    Book(int bookId, String bookName, String author, double price) {
        this.bookId = bookId;
        this.bookName = bookName;
        this.author = author;
        this.price = price;
    }
    void method2(){
        System.out.println("Book ID: " + bookId);
        System.out.println("Book Name: " + bookName);
        System.out.println("Author: " + author);
        System.out.println("Price: " + price);
        System.out.println();
    }
}

class Book1{
    private int bookId;
    private String bookName;
    private String author;
    private double price;

    //constructor
    Book1(){
        bookId=0;
        bookName="unknown";
        author="unknown";
        price=0.00;
    }

    //constructor with parameters
    Book1(int id,String name,String aut,double p){
        bookId=id;
        bookName=name;
        author=aut;
        price=p;
    }

    // getters & setters for Encapsulation
    public int getBookId(){
        return bookId;
    }
    public void setBookId(int id){
        bookId = id;
    }

    public String getBookName(){
        return bookName;
    }
    public void setBookName(String name){
        bookName = name;
    }

    public String getAuthor(){
        return author;
    }
    public void setAuthor(String aut){
        author = aut;
    }

    public double getPrice(){ return price;
    }
    public void setPrice(double p){
        price = p;
    }

    public void details() {
        System.out.println(bookId + " | " + bookName + " | " + author + " | " + price);
    }
}

// inheritance super:person sub:student and faculty
class Person {
    void method1(){
        System.out.println("output from class person");
    }
}
```

```

class Student extends Person {
    void method2(){
        System.out.println("output from class student which inherit person");
    }
}

class Faculty extends Person {
    void method3(){
        System.out.println("output from class faculty which inherits person");
    }
}

//polymorph
class AreaCalculator {
    double area(double radius) {
        return Math.PI * radius * radius;
    }
    double area(double length, double breadth) {
        return length * breadth;
    }
    double area(double base, double height, boolean triangle) {
        return 0.5 * base * height;
    }
}

class Vehicle {
    void display() {
        System.out.println("This is a vehicle.");
    }
}

class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Car.");
    }
}

class Bike extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Bike.");
    }
}

// abstraction
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

interface Printable {
    void print();
}

class Report implements Printable {
    public void print() {
        System.out.println("Printing Report...");
    }
}

public class Task2{
    public static void main(String[] args){
        Book b1 = new Book(101, "Java Programming", "James Gosling", 450);
        // Book b2 = new Book(102, "Data Structures", "Mark Allen", 550);
        b1.method2();

        Book1 obj=new Book1(298137,"javaclass","nani",20.3);
        obj.details();

        Student obj1=new Student();
        obj1.method1();
        obj1.method2();

        Faculty obj2=new Faculty();
    }
}

```

```

        obj2.method1();
        obj2.method3();

        Vehicle v = new Vehicle();
        Vehicle car = new Car();
        Vehicle bike = new Bike();
        v.display();
        car.display();
        bike.display();

        Shape circle = new Circle();
        Shape rectangle = new Rectangle();
        circle.draw();
        rectangle.draw();
        Report report = new Report();
        report.print();
    }
}

```

■ Compilation & Execution Commands

Save as: Task2.java

```

javac Task2.java
java Task2

```

■■ Output Screen

C:\Java>javac Task2.java

```

C:\Java>java Task2
Book ID: 101
Book Name: Java Programming
Author: James Gosling
Price: 450.0

```

```

298137 | javaclass | nani | 20.3
output from class person
output from class student which inherit person
output from class person
output from class faculty which inherits person
This is a vehicle.
This is a Car.
This is a Bike.
Drawing a Circle
Drawing a Rectangle
Printing Report...

```

C:\Java>

■ Concepts Demonstrated

Concept	Demonstrated By
Constructors	Book and Book1
Encapsulation	Book1 private fields + getters/setters
Inheritance	Person → Student and Faculty
Polymorphism	AreaCalculator overloading; Vehicle overriding
Abstraction	Shape → Circle and Rectangle
Interface	Printable → Report

■ Result

The Java program was successfully compiled and executed. It demonstrates constructors, encapsulation, inheritance, polymorphism, abstraction, and interfaces using the supplied Task2.java source code.